

Practice Sheet 1

SECTION 02

Warm-Up Questions

1. What is the difference between == and is in Python? Give one example where they produce different results.

Answer -

- == Operator (Equality Operator): It is used when you want to check whether two objects contain same data, regardless of whether they are stored in same memory location.
- is Operator (Identity Operator): The is operator is used when you want to check whether two variables refer to exact same object in memory. It does not care about values.

2. Why does `0.1 + 0.2 == 0.3` return False ? What is the correct way to compare floats?

Answer -This is because computers store numbers using the IEEE 754 standard for floating-point arithmetic.Because hardware allocates a fixed size (usually 64 bits for double-precision), the computer truncates these infinite sequences and rounds them. When you add the rounded binary representations of 0.1 and 0.2, the calculation yields a small rounding error. The result evaluates to 0.30000000000000004. Since 0.30000000000000004 does not exactly match 0.3, the exact equality operator (==) evaluates to False. The correct way to compare floats is to use `math.isclose()`

```
<code = import math
```

```
math.isclose(0.1 + 0.2, 0.3) # Returns True
```

3. Explain the LEGB rule. In what order does Python resolve a variable name?

Answer - The LEGB rule is used to decide the order in which the namespaces are to be searched for scope resolution. The order in which Python resolve a variable name is:

- Local(L): Defined inside function/class
- Enclosed(E): Defined inside enclosing functions(Nested function concept)
- Global(G): Defined at the uppermost level
- Built-in(B): Reserved names in Python builtin modules

4. What is a mutable default argument bug? How do you fix it?

Answer - The mutable default argument bug is a famous Python "gotcha" that occurs when you use a mutable object—like a list ([]), dictionary ({}), or set—as a default parameter value in a function definition. The standard solution is to use None as the default value and create a new mutable object inside the function.

5. What does `and` return when all values are truthy? What does `or` return when all values are falsy?

Answer - The `and` operator returns the last truthy value if all values are truthy. The `or` operator returns the last falsy value if all values are falsy.

6. What is the difference between `//` and `/` ? What does `-7 // 2` return and why?

Answer - `/` Operator: This operator is used for true division. No matter what the inputs are, result is always a floating-point number. This operator preserves fractional part of the result. `//` Operator: This operator is used for floor division. Instead of keeping fractional part, it returns largest integer less than or equal to the result. `-7 // 2` will return `-4` because the `//` operator does not simply truncate the decimal; it finds the largest integer less than or equal to the result.

7. What does `for-else` do in Python? When does the `else` block run?

Answer -The `for else` loop in Python allows you to execute a block of code when the `for` loop completes its iteration normally (i.e., without encountering a `break` statement). The `else` block run only if the `for` loop finishes iterating over the sequence without being interrupted by a `break` statement.

8. Why should you never use `str +=` inside a loop? What should you use instead?

Answer -`str +=` inside a loop is generally an anti-pattern because it results in quadratic running time $O(n^2)$ and excessive memory consumption. Instead of concatenating with `+=`, append every substring to a list and then use the built-in `"".join()` method. `.join()` calculates the total size required for the new string before creating it.

9. What is the difference between a generator and a list comprehension?

Answer- Generators: These yield elements on demand one by one. Generators are far more memory-efficient for large or infinite datasets. List comprehension: These create a new list in memory immediately, storing all elements at once. List comprehensions are faster and more flexible for smaller datasets.

10. What does `@functools.wraps(func)` do, and why is it mandatory in decorators?

Answer - `@functools.wraps(func)` is a helper decorator used inside custom decorators to copy important metadata from the original function to the newly returned wrapper function. It prevents a decorator from permanently overwriting a wrapped function's identity and metadata.

Rapid-Fire Questions

1. What does `type(True)` return?

Answer- It returns `<class 'bool'>` as it belongs to the Python boolean data type.

2. Is `bool` a subclass of `int` in Python?

Answer- Yes, `bool` is a explicit subclass of `int` in Python.

3. What is the output of `True + True + True` ?

Answer- The output of `True + True + True` is `3`.

4. What Python version introduced the walrus operator `:=` ?

Answer- The walrus operator `:=` was introduced in Python 3.8.

5. What Python version introduced `match-case` ?

Answer- The `match-case` statement was introduced in Python 3.10.

6. What is the CPython integer cache range?
Answer- CPython integer cache ranges from -5 to 256.
7. What does id() return?
Answer- The memory address of an object.
8. Name three falsy values in Python.
Answer- None, False and 0.
9. What operator checks membership in a collection?
Answer- in operator checks membership in a collection.
10. What does **kwargs capture in a function?
Answer- A dictionary of keyword arguments.

Fill in blanks

1. `abs(a - b) < 1e-9`
2. `a, b = b, a`
3. if result is None:
4. `def append_item(item, lst=None): if lst is None: lst = []`
5. `result = ".join(words)`
6. `while chunk := f.read(1024):`
7. `@functools.wraps(func)`
8. -5
9. `for i, v in enumerate(my_list):`
10. 'and' returns the first falsy value, or the last value if all are truthy.

SECTION 03

Predict the Output

Easy Level

```
In [3]: # E1 - Your prediction: 3
#          1
#          3.33333333333333
x = 10
y = 3
print(x // y)
print(x % y)
print(x / y)
```

```
3
1
3.333333333333335
```

```
In [5]: # E2 - prediction: @LICE
name = " Alice "
print(name.strip().upper().replace('A', '@'))
```

```
@LICE
```

```
In [3]: # E3 - prediction: 1, 2, 3, 4
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

[1, 2, 3, 4]

```
In [ ]: # E4 - prediction: False True False True False
print(bool(""), bool(" "), bool(0), bool([0]), bool(None))
```

Intermediate level

```
In [ ]: ## I1 - Your prediction: _____
print(0 and "hello") # prediction: 0
print("" or "default") # prediction: default
print(None or [] or 0) # prediction: 0
print("a" and "b" and "c") # prediction: c
```

```
In [ ]: # I2 - prediction: positive
#               negative
x = 5
result = "positive" if x > 0 else "negative" if x < 0 else "zero"
print(result)
y = -3
result2 = "positive" if y > 0 else "negative" if y < 0 else "zero"
print(result2)
```

```
In [10]: # I3 - Your prediction: World
#               !dlrow ,olleH
#               Lo ol!
s = "Hello, World!"
print(s[7:12])
print(s[::-1])
print(s[:2])
```

World
!dlrow ,olleH
Hlo ol!

```
In [11]: # I4 - Your prediction: 6
#               9
#               20
def f(x, multiplier=2):
    return x * multiplier
print(f(3))
print(f(3, 3))
print(f(multiplier=5, x=4))
```

6
9
20

Tricky level

```
In [ ]: # T1 - Your prediction: -4
#               1
#               -4
```

```
# -1
print(-7 // 2)
print(-7 % 2)
print(7 // -2)
print(7 % -2)
```

```
In [13]: # T2 - Your prediction: 64
# 512
# prediction is not accurate because of wrong associativity.
print(2 ** 3 ** 2) # right-associative exponentiation
print((2 ** 3) ** 2)
```

512

64

```
In [15]: # T3 - Your prediction: ['python']
# ['python', 'ml']
# ['python', 'ml', 'ai']_
def add_tag(item, tags=[]):
    tags.append(item)
    return tags
print(add_tag("python"))
print(add_tag("ml"))
print(add_tag("ai"))
```

['python']

['python', 'ml']

['python', 'ml', 'ai']

```
In [16]: # T4 - Your prediction: [2, 2, 2]
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])
```

[2, 2, 2]

Hidden behavior questions

```
In [17]: # H1 - Integer caching: Your prediction: True
# True --> this prediction is wrong because
a = 256
b = 256
print(a is b)
c = 257
d = 257
print(c is d)
```

True

False

```
In [ ]: # H2 - Boolean arithmetic: Your prediction: 2
# 10
# 3
# 1 0
print(True + True)
print(True * 10)
print(sum([True, False, True, True, False]))
print(int(True), int(False))
```

```
In [ ]: # H3 – for-else: Your prediction: All even
#                                             Found odd

for n in [2, 4, 6, 8]:
    if n % 2 != 0:
        print("Found odd")
        break
    else:
        print("All even")
for n in [2, 3, 6, 8]:
    if n % 2 != 0:
        print("Found odd")
        break
    else:
        print("All even")
```

SECTION 04

Debugging Challenge Zone

Syntax Errors

```
In [ ]: # SYN-1: Find and fix all syntax errors
# DON'T RUN yet – find the bugs by reading first
def greet(name
    message = f"Hello, {name}!"
    return message
# Your fix below: def greet(name):
```

```
In [ ]: # SYN-2: This code has 2 syntax errors
for i in range(5)
    if i % 2 = 0:
        print(i, "is even")
# Your fix below: for i in range(5):
#                     if i % 2 == 0:
```

Logical errors

```
In [ ]: # LOG-1: Fee calculator (classic production bug)
# Intent: 2% fee, minimum ₹5, maximum ₹200
def calculate_fee(amount):
    fee = amount * 2 / 100
    if fee < 5:
        fee == 5 # BUG: What is wrong here?
    if fee > 200:
        fee == 200 # BUG: And here?
    return fee
print(calculate_fee(100)) # Expected: 5 (minimum applies)
print(calculate_fee(5000)) # Expected: 100
print(calculate_fee(15000)) # Expected: 200 (maximum applies)
# Identify bugs:
In [ ]:
In [ ]:
In [ ]:
# Bug 1: fee == 5
```

```
# Bug 2:
# Your fix: fee = 5
#           fee = 200
```

```
In [ ]: # LOG-2: Grade classifier – why does it always print "F"?
def classify_grade(score):
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    elif score >= 70:
        grade = "C"
    elif score >= 60:
        grade = "D"
    grade = "F" # BUG: Where is the error?
    return grade
print(classify_grade(95)) # Should be A
print(classify_grade(75)) # Should be C
print(classify_grade(45)) # Should be F
# Explanation: default case is always written inside an else:, otherwise it will
# Your fix: else:
#           grade = "F"
```

```
In [ ]: # LOG-3: Count vowels function – returns wrong count
def count_vowels(text):
    count = 0
    for char in text:
        if char in "aeiou": # BUG: What's missing?
            count += 1
    return count
print(count_vowels("Hello World")) # Expected: 3
print(count_vowels("APPLE")) # Expected: 2
In [ ]:
In [ ]:
# Explanation: The condition if char in "aeiou": only checks lowercase vowels,
# Your fix: if char.lower() in "aeiou":
```

Runtime errors

```
In [ ]: # RT-1: What error does this raise? Fix it.
# def divide(a, b):
#     return a / b
# results = [divide(10, x) for x in [2, 5, 0, 1]]
# print(results)
# Error type: Zero division error[when x= 0, 10/0 will raise runtime error]
# Your fix (handle gracefully, return None for invalid inputs):
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None
results = [divide(10, x) for x in [2, 5, 0, 1]]
print(results) # [5.0, 2.0, None, 10.0]
```

```
In [ ]: # RT-2: What error does this raise? Fix it.
# data = {"name": "Alice", "age": 25}
# print(f"Name: {data['name']}, Score: {data['score']}")
# Error type: the dictionary data does not contain a key named 'score'
```

```
# Your fix (use .get() with a default):
data = {"name": "Alice", "age": 25}
print(f"Name: {data['name']}, Score: {data.get('score', 'N/A')}")
```

```
In [ ]: # RT-3: UnboundLocalError – explain why and fix it
# counter = 0
# def increment():
#     counter += 1 # BUG: Why does this fail?
#     return counter
# increment()
# Explanation: Python sees counter += 1 as an assignment. Python raises UnboundL
# Your fix (two solutions: global keyword OR return value approach):
def increment(counter):
    return counter + 1
counter = 0
counter = increment(counter)
print(counter) # 1
counter = increment(counter)
print(counter) # 2
```

Find the Hidden Issue

```
In [ ]: # HIDDEN-1: This runs without error but produces WRONG results
# Intent: Remove all negative numbers from the List
numbers = [1, -2, 3, -4, 5, -6, 7]
for num in numbers:
    if num < 0:
        numbers.remove(num)
print(numbers) # What do you get? What should you get?
# Why does it fail? ==> because modifying the list while iterating over it will
# Your fix:
# numbers = [1, -2, 3, -4, 5, -6, 7]
# numbers = [num for num in numbers if num >= 0]
# print(numbers) # [1, 3, 5, 7]
```

```
In [ ]: # HIDDEN-2: Closure late-binding bug
# Intent: Create 3 functions that return 0, 1, 2 respectively
# multipliers = []
# for i in range(3):
#     multipliers.append(lambda x: x * i)
# print([f(10) for f in multipliers]) # Expected: [0, 10, 20]. Actual?
# Why does it fail? ==> All three lambdas reference the same variable i, as the l
# Fix 1 (default arg trick):
multipliers = []
for i in range(3):
    multipliers.append(lambda x, i=i: x * i)
print([f(10) for f in multipliers]) # [0, 10, 20]

# Fix 2 (using functools.partial or comprehension):
multipliers = [lambda x, i=i: x * i for i in range(3)]
print([f(10) for f in multipliers]) # [0, 10, 20]
```

SECTION 05

Edge Cases & Hidden Scenarios

```

In [23]: # EC-1: EMPTY INPUT CASES
# The function below works for normal input – make it handle all edge cases
def average(numbers):
    """Return the average of a list of numbers."""
    return sum(numbers) / len(numbers)
# Test cases to handle:
# average([]) → should return None or raise a clear ValueError
# average([5]) → should return 5.0
# average(None) → should raise TypeError with clear message
# average(["a", "b"]) → should raise TypeError
# average([1, 2, None, 4]) → handle None in List
# Your improved version:
# def average(numbers):
#     """Return the average of a list of numbers, handling edge cases."""
#     if numbers is None:
#         raise TypeError("Input must be a list of numbers, not None")
#     if not isinstance(numbers, list):
#         raise TypeError("Input must be a list of numbers")
#     if len(numbers) == 0:
#         return None # or: raise ValueError("Cannot compute average of empty L
#     # Filter out None values
#     cleaned = [n for n in numbers if n is not None]
#     # Ensure all remaining elements are numeric
#     if not all(isinstance(n, (int, float)) for n in cleaned):
#         raise TypeError("List must contain only numbers")
#     return sum(cleaned) / len(cleaned)
# print(average([]))
# print(average([5]))
# print(average(None))
# print(average(["a", "b"]))
# print(average([1, 2, None, 4]))

```

```

In [ ]: # EC-2: BOUNDARY CONDITIONS
# Check if a password meets requirements:
# Length: 8-20 chars, has uppercase, lowercase, digit, special char
def is_valid_password(pwd):
    # Test these boundaries:
    # "" → False (empty)
    # "abc" → False (too short)
    # "a" * 21 → False (too long)
    # "Abcdefg1!" → True
    # "ABCDEFG1!" → False (no lowercase)
    # "abcdefg1!" → False (no uppercase)
    # "Abcdefgh!" → False (no digit)
    # "Abcdefg12" → False (no special char)
    pass
# Your implementation:
import string
def is_valid_password(pwd):
    """Check if a password meets requirements:
    - Length: 8-20 characters
    - Contains uppercase, lowercase, digit, and special character
    """
    if not isinstance(pwd, str):
        return False
    # Length check
    if len(pwd) < 8 or len(pwd) > 20:
        return False
    # Flags for each requirement

```

```

has_upper = any(ch.isupper() for ch in pwd)
has_lower = any(ch.islower() for ch in pwd)
has_digit = any(ch.isdigit() for ch in pwd)
has_special = any(ch in string.punctuation for ch in pwd)
return has_upper and has_lower and has_digit and has_special
print(is_valid_password("")) # False (empty)
print(is_valid_password("abc")) # False (too short)
print(is_valid_password("a"*21)) # False (too long)
print(is_valid_password("Abcdefg1!")) # True
print(is_valid_password("ABCDEFG1!")) # False (no lowercase)
print(is_valid_password("abcdefg1!")) # False (no uppercase)
print(is_valid_password("Abcdefgh!")) # False (no digit)
print(is_valid_password("Abcdefg12")) # False (no special char)

```

```

In [ ]: # EC-4: LARGE DATA SCENARIO
# The code below works for small lists. What happens at scale? Rewrite it.
# Original (inefficient):
def find_duplicates_slow(items):
    """Find items that appear more than once."""
    duplicates = []
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j] and items[i] not in duplicates:
                duplicates.append(items[i])
    return duplicates
# Questions:
# 1. What is the time complexity of this function?  $O(n^2)$ 
# 2. What is the space complexity?  $O(n)$ 
# 3. At 1 million items, roughly how many comparisons?  $\Rightarrow (n(n-1))/2$ 
# 4. Rewrite it with  $O(n)$  time using a set/dict:
def find_duplicates(items):
    """Find items that appear more than once in  $O(n)$  time."""
    seen = set()
    duplicates = set()
    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

```

SECTION 06

Industry-Based Scenarios

BANKING SYSTEM

```

In [ ]: def calculate_upi_fee(amount):
    """
    Calculate UPI transaction fee.
    Returns: fee as float, "free" for below ₹100, or None if input is invalid.
    """
    if isinstance(amount, str):
        print(f"Invalid input: string '{amount}'")
        return None
    if not isinstance(amount, (int, float)):

```

```

        print(f"Invalid input type: {type(amount)}")
        return None
    if amount <= 0:
        print(f"Invalid amount: {amount}")
        return None
    if amount < 100:
        return "free"
    # Calculate fee
    fee = round(amount * 0.018, 2)
    if fee < 2:
        fee = 2.0
    if fee > 1000:
        fee = 1000.0

    if isinstance(amount, float) and not amount.is_integer():
        print(f"Invalid float amount: {amount}")
        return None
    return fee

# Test cases
test_cases = [
    (50, "free"),          # Below ₹100
    (100, 2.0),            # Minimum fee
    (500, 9.0),            # Normal: 1.8% of 500
    (70000, 1000.0),       # Maximum fee cap
    (499.99, None),        # Float input
    ("500", None),         # String input
    (-100, None),          # Negative – invalid
    (0, None),             # Zero – invalid
]

for amount, expected in test_cases:
    result = calculate_upi_fee(amount)
    status = "✓" if result == expected else "X"
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")

```

#HEALTHCARE SYSTEM

In [8]: # HEALTHCARE: Implement calculate_bmi(weight_kg, height_m) → dict | None

```

def calculate_bmi(weight_kg, height_m):
    """
    Calculate BMI and classify health risk.
    Returns dict with bmi, category, risk_level – or None if invalid.
    """
    try:
        # Validate inputs: must be numeric
        if weight_kg is None or height_m is None:
            print("Error: weight or height is None")
            return None

        # Convert string numbers if possible
        if isinstance(weight_kg, str):
            if weight_kg.isdigit():
                weight_kg = float(weight_kg)
            else:
                print("Error: weight must be numeric")
                return None
        if isinstance(height_m, str):
            if height_m.replace(".", "", 1).isdigit():

```

```

        height_m = float(height_m)
    else:
        print("Error: height must be numeric")
        return None

    # Ensure numeric types
    if not isinstance(weight_kg, (int, float)) or not isinstance(height_m, (
        print("Error: inputs must be int or float")
        return None

    # Validate ranges
    if not (1 <= weight_kg <= 500):
        print("Error: weight out of range (1-500 kg)")
        return None
    if not (0.5 <= height_m <= 2.5):
        print("Error: height out of range (0.5-2.5 m)")
        return None

    # Calculate BMI
    bmi = weight_kg / (height_m ** 2)
    bmi = round(bmi, 1)

    # Classification
    if bmi < 18.5:
        category, risk = "Underweight", "Moderate"
    elif 18.5 <= bmi <= 24.9:
        category, risk = "Normal", "Low"
    elif 25 <= bmi <= 29.9:
        category, risk = "Overweight", "Elevated"
    else:
        category, risk = "Obese", "High"

    return {"bmi": bmi, "category": category, "risk_level": risk}

except Exception as e:
    print(f"Error: {e}")
    return None

# Test cases:
print(calculate_bmi(70, 1.75))      # Normal: ~22.9
print(calculate_bmi(50, 1.75))      # Underweight: ~16.3
print(calculate_bmi(100, 1.75))     # Obese: ~32.7
print(calculate_bmi(-5, 1.75))      # Invalid
print(calculate_bmi(70, 0))          # Invalid (division by zero risk)
print(calculate_bmi("seventy", 1.75))# Invalid

```

```

{'bmi': 22.9, 'category': 'Normal', 'risk_level': 'Low'}
{'bmi': 16.3, 'category': 'Underweight', 'risk_level': 'Moderate'}
{'bmi': 32.7, 'category': 'Obese', 'risk_level': 'High'}
Error: weight out of range (1-500 kg)
None
Error: height out of range (0.5-2.5 m)
None
Error: weight must be numeric
None

```

SECTION 07

Think Like an Engineer

Scenario A: A function `get_user_data(user_id)` fetches a user's data from a database and returns a dict.

1. What could go wrong? List at least 5 failure modes.

- Invalid input: `user_id` is `None`, negative, or of the wrong type (string instead of int).
- User not found: Query returns no rows, leading to empty or `None` results.
- Database connectivity issues: Timeout, server crash, or network failure.
- Data corruption: Missing fields, unexpected nulls, or malformed values in the database.
- Authorization errors: Caller lacks permission to access the requested user's data.

2. What assumptions is the current design making?

- `user_id` provided is always valid and exists in the database.
- Database connection is reliable and available.
- Returned data is complete and correctly structured as a dict.
- Caller has permission to access the user's data.

3. How would you validate inputs before the DB call?

- Type check: Ensure `user_id` is an integer (or UUID if that's the design).
- Range check: Reject negative or zero IDs if not valid.
- Null check: Ensure `user_id` is not `None`.
- Sanitize inputs: Prevent SQL injection if raw queries are used.

4. How would this behave at scale? (1 million calls/hour)

- Performance bottlenecks: Database may not handle that many queries without optimization.
- Connection pool exhaustion: Too many concurrent requests overwhelm DB connections.
- Latency spikes: Response times increase under heavy load.
- Caching required: Frequently accessed user data should be cached (Redis, Memcached).
- Sharding/replication: Database may need horizontal scaling to distribute load.

5. What business impact could failures cause?

- Customer dissatisfaction: Users unable to access their account details.
- Financial loss: Failed transactions or incorrect balances.
- Compliance risks: Missing or incorrect data could violate regulations.
- Operational disruption: Support teams flooded with complaints.

Scenario B: A loop processes 10,000 order records. For each order, it builds a report string using `+=`

1. What is the time complexity of string += in a loop? Why? In Python, strings are immutable. Each += creates a new string object, copying the old content plus the new addition. So, For n concatenations, this results in $O(n^2)$ time complexity because each step copies increasingly larger strings.
2. How much memory could this consume for 10,000 records of 1KB each?
3. What is the correct approach? Correct Approach : Use a list to collect fragments, then join once at the end: parts = [] for order in orders:
parts.append(build_report(order)) report = "".join(parts)
4. How would you test that your fix actually improved performance?
 - Benchmarking: Use timeit to measure execution time of += vs join. Memory profiling: Use tracemalloc or memory_profiler to track memory usage.
 - Load testing: Run with realistic dataset sizes (10k, 100k records) and compare runtime/memory.
 - Regression tests: Ensure output correctness is identical between old and new implementations.

SECTION 08

Hands-On Coding Practice

— BEGINNER

```
In [26]: # B1: Write a function that takes a temperature in Celsius and returns it in
# Fahrenheit AND Kelvin. Return as a tuple: (fahrenheit, kelvin)
# Formula: F = (C × 9/5) + 32 | K = C + 273.15
def convert_temperature(celsius):
    """
    Convert Celsius to Fahrenheit and Kelvin.
    Returns a tuple: (fahrenheit, kelvin)
    """
    fahrenheit = (celsius * 9/5) + 32
    kelvin = celsius + 273.15
    return (float(fahrenheit), float(kelvin))

# Tests:
assert convert_temperature(0) == (32.0, 273.15)
assert convert_temperature(100) == (212.0, 373.15)
print("B1 passed ")
```

B1 passed

```
In [ ]: # B2: Write a function that checks if a string is a palindrome
# Ignore case and spaces
# "A man a plan a canal Panama" → True
# "hello" → False
def is_palindrome(text):
    """
    Check if a string is a palindrome.
    Ignores case and spaces.
    """
```

```

# Normalize: Lowercase + remove spaces
cleaned = "".join(ch.lower() for ch in text if ch.isalnum())
return cleaned == cleaned[::-1]

# Tests:
assert is_palindrome("racecar") == True
assert is_palindrome("hello") == False
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("") == True
print("B2 passed ")

```

— INTERMEDIATE

```

In [14]: # I1: Write a decorator `timer` that measures and prints execution time
# Use functools.wraps. The print format: "<func_name> took X.XXXs"
import functools
import time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        elapsed = end - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    """Sum numbers from 0 to n."""
    return sum(range(n))

# Run test
result = slow_sum(10_000_000)
print(f"Result: {result}")
print(f"Function name preserved: {slow_sum.__name__}") # Must print 'slow_sum'

```

```

slow_sum took 0.1634s
Result: 49999995000000
Function name preserved: slow_sum

```

```

In [15]: # I3: Generator for Fibonacci sequence
# Write a generator function fibonacci(n) that yields the first n Fibonacci numb
# Then use it to: find the first Fibonacci number > 1000
def fibonacci(n):
    """
    Generator that yields the first n Fibonacci numbers.
    """
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

```

```
# Tests:
fib_list = list(fibonacci(10))
assert fib_list == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34], f"Got {fib_list}"

# Find first Fibonacci > 1000 using generator expression:
first_over_1000 = next(x for x in fibonacci(100) if x > 1000)

print(f"First Fibonacci > 1000: {first_over_1000}")
print("I3 passed ")
```

First Fibonacci > 1000: 1597

I3 passed

— ADVANCED

```
In [16]: # A1: Implement a memoization decorator from scratch (without functools.lru_cache)
# The decorator should cache function results based on arguments
# Add a .cache_info() method that returns {hits, misses, size}
def memoize(func):
    cache = {}
    hits = 0
    misses = 0

    def wrapper(*args, **kwargs):
        nonlocal hits, misses
        key = (args, tuple(sorted(kwargs.items())))
        if key in cache:
            hits += 1
            return cache[key]
        else:
            result = func(*args, **kwargs)
            cache[key] = result
            misses += 1
            return result

    def cache_info():
        return {"hits": hits, "misses": misses, "size": len(cache)}

    wrapper.cache_info = cache_info
    return wrapper

@memoize
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

# Test
print(fib(30))          # Should be fast due to memoization
print(fib.cache_info()) # Should show hits, misses, size
```

832040

{'hits': 28, 'misses': 31, 'size': 31}

SECTION 09

Scenario-Based Coding Task

```
In [18]: # SCENARIO 1: Student Report Card Generator
# You receive student data as a List of dicts:
# [{"name": "Alice", "marks": {"math": 85, "science": 92, "english": 78}}, ...]
#
# Build a function generate_report(students) that returns a List of dicts with:
# - name, total, average (2 decimal places), grade (A/B/C/D/F), rank (1=highest)
# - Handle: missing subjects, marks outside 0-100, empty students list
# - Sort output by rank (highest average first)
students = [
    {"name": "Alice", "marks": {"math": 85, "science": 92, "english": 78}},
    {"name": "Bob", "marks": {"math": 72, "science": 68, "english": 80}},
    {"name": "Charlie", "marks": {"math": 95, "science": 88, "english": 91}},
    {"name": "Priya", "marks": {"math": 60, "science": 55, "english": 70}},
]

def generate_report(students):
    if not students:
        return []

    report = []
    for student in students:
        name = student.get("name", "Unknown")
        marks = student.get("marks", {})

        # Filter valid marks (0-100)
        valid_scores = [
            score for score in marks.values()
            if isinstance(score, (int, float)) and 0 <= score <= 100
        ]

        if not valid_scores:
            total = 0
            avg = 0.0
        else:
            total = sum(valid_scores)
            avg = round(total / len(valid_scores), 2)

        # Grade classification
        if avg >= 90:
            grade = "A"
        elif avg >= 80:
            grade = "B"
        elif avg >= 70:
            grade = "C"
        elif avg >= 60:
            grade = "D"
        else:
            grade = "F"

        report.append({
            "name": name,
            "total": total,
            "average": avg,
            "grade": grade
        })

    # Sort by average descending
```

```

report.sort(key=lambda x: x["average"], reverse=True)

# Assign ranks
for idx, rec in enumerate(report, start=1):
    rec["rank"] = idx

return report

# Run and print
for record in generate_report(students):
    print(record)

```

```

{'name': 'Charlie', 'total': 274, 'average': 91.33, 'grade': 'A', 'rank': 1}
{'name': 'Alice', 'total': 255, 'average': 85.0, 'grade': 'B', 'rank': 2}
{'name': 'Bob', 'total': 220, 'average': 73.33, 'grade': 'C', 'rank': 3}
{'name': 'Priya', 'total': 185, 'average': 61.67, 'grade': 'D', 'rank': 4}

```

SECTION 10

Production Thinking Challenges

Scalability

1. The following function is called 10,000 times per second in production. What would you change?

```

def get_tax_rate(country):
    rates = {"IN": 18, "US": 8, "UK": 20, "DE": 19}
    return rates.get(country, 0)

```

Your answer: The rates dictionary is recreated on every function call, which wastes CPU time and memory. Move it outside the function so it is created only once.

```

RATES = {"IN": 18, "US": 8, "UK": 20, "DE": 19}
def get_tax_rate(country):
    return RATES.get(country, 0)

```

2. A function builds a report for each of 500,000 users. It currently uses a list to accumulate results and appends one item per user. What is the memory concern, and what is the solution?

Your answer: Storing 500,000 report items in a list can consume a large amount of memory and may lead to memory issues.

The solution is to use a generator (yield) or process/write each report item incrementally instead of storing everything in memory at once.

```

def generate_reports(users):
    for user in users:
        yield build_report(user)

```

Reliability

3. What is wrong with this error handling?

try:

result = process_payment(order)

except:

pass

Your answer: The except: pass block silently ignores all errors, making failures difficult to detect, debug, and monitor. It can hide critical issues such as payment failures. 4. A function calls an external API to get exchange rates. What reliability measures should you add?

Your answer:

- Add timeouts to prevent hanging requests.
- Add retry logic for temporary failures.
- Handle and log errors properly.

SECTION 11

Code Improvement Exercises

IMPROVE-1: Refactor for readability, efficiency, and edge case handling

```
In [22]: # The code works but is bad in every professional sense.
# def f(L):
#     r = []
#     for i in range(0, len(L)):
#         if L[i] > 0:
#             r = r + [L[i] * 2]
#     return r
# Problems to fix:
# 1. No docstring / type hints
# 2. Bad variable names (l looks like 1)
# 3. r = r + [item] is O(n²) – should use append or comprehension
# 4. range(0, len(L)) – antipattern, just iterate directly
# 5. No edge case handling (None input, non-numeric items)
# Your improved version:
from typing import Iterable, List

def double_positive_numbers(numbers: Iterable[float]) -> List[float]:
    """
    Return a list containing double the value of all positive numbers.
    """
    if numbers is None:
        raise ValueError("Input cannot be None")
    result = []
    for item in numbers:
        if not isinstance(item, (int, float)):
            raise TypeError(f"Non-numeric item found: {item}")

        if item > 0:
            result.append(item * 2)
```

```

    return result
# Test the function
numbers = [1, -2, 3, 0, 5]
output = double_positive_numbers(numbers)
print("Input:", numbers)
print("Output:", output)

```

IMPROVE-2: Fix the mutable default, add type hints, handle edge cases

```

In [23]: # def add_to_cart(item, cart=[]):
#         cart.append(item)
#         print(f"Cart: {cart}")
#         return cart
# # Problems:
# # 1. Mutable default argument – cart persists across calls
# # 2. No input validation
# # 3. No type hints or docstring
# # 4. Mixes side effects (print) with return value
# # Demonstrate the bug first:
# cart1 = add_to_cart("apple")
# cart2 = add_to_cart("banana") # Should start fresh!
# print("Bug: cart1 is cart2:", cart1 is cart2)
# Your improved version:
from typing import List, Optional
# Demonstrating the bug
def add_to_cart(item, cart=[]):
    cart.append(item)
    print(f"Cart: {cart}")
    return cart
cart1 = add_to_cart("apple")
cart2 = add_to_cart("banana") # Should start fresh!
print("Bug: cart1 is cart2:", cart1 is cart2)
# Improved version
def add_to_cart_fixed(item: str, cart: Optional[List[str]] = None) -> List[str]:
    """
    Add an item to a shopping cart and return the updated cart.
    Args:
        item: Item to add.
        cart: Existing cart or None to create a new cart.
    Returns:
        Updated cart list.
    """
    if not item:
        raise ValueError("Item cannot be empty")
    if cart is None:
        cart = []
    cart.append(item)
    return cart

# Testing the improved version
cart1 = add_to_cart_fixed("apple")
cart2 = add_to_cart_fixed("banana")

print("Cart 1:", cart1)
print("Cart 2:", cart2)
print("cart1 is cart2:", cart1 is cart2)

```

```

Cart: ['apple']
Cart: ['apple', 'banana']
Bug: cart1 is cart2: True
Cart 1: ['apple']
Cart 2: ['banana']
cart1 is cart2: False

```

SECTION 11

Interview Preparation

INTERVIEW CODING ROUND : Common live-coding question

```

In [25]: ## Write a function that takes a string of words and returns the word
          ## that appears most frequently. If there's a tie, return all tied words
          ## in alphabetical order. Ignore case. Ignore punctuation.
          ##
          ## Example:
          ## "the cat sat on the mat the" → "the" (appears 3 times)
          ## "cat bat cat bat dog" → ["bat", "cat"] (tie)
          # def most_frequent_word(text):
          #     pass
          # print(most_frequent_word("the cat sat on the mat the"))
          # print(most_frequent_word("cat bat cat bat dog"))
          # print(most_frequent_word("")) # Edge case
          # Solution :
import re
from collections import Counter
def most_frequent_word(text):
    # Edge case: empty string
    if not text:
        return None
    # Convert to Lowercase and remove punctuation
    words = re.findall(r'\b\w+\b', text.lower())
    if not words:
        return None
    # Count word frequencies
    counts = Counter(words)
    # Find highest frequency
    max_count = max(counts.values())
    # Get all words with highest frequency
    most_common = sorted(
        [word for word, count in counts.items() if count == max_count]
    )
    # Return single word or list of tied words
    return most_common[0] if len(most_common) == 1 else most_common

# Test cases
print(most_frequent_word("the cat sat on the mat the"))
print(most_frequent_word("cat bat cat bat dog"))
print(most_frequent_word("")) # Edge case

```

```

the
['bat', 'cat']
None

```

In []: